

Keywords: KV cache • sparse attention • long-context inference • attention compensation

CompKV

Compensation-Aware KV Selection for Long-Context LLM Inference

By Anonymous et al.

arXiv:2609.26300 • September 2026

By pairing block-level KV selection with mean compensation—which yields an exact log-partition identity—CompKV derives a selection priority proportional to block attention mass times within-block logit variation, consistently outperforming prior sparse-attention methods on RULER and LongBench-Pro at tight token budgets.

AT A GLANCE

Problem: Sparse-attention KV selection methods discard unselected context entirely, introducing approximation error that biases the softmax partition function.

Why it matters: Long-context tasks requiring global aggregation fail most under naive selection; more principled compensation preserves performance at aggressive memory budgets.

Method: Mean compensation for unselected blocks yields an exact log-partition identity; the resulting optimal selection priority is block attention mass $\times$ within-block logit variation.

Result: CompKV consistently beats selection-only and eviction-based sparse attention on Llama-3.1-8B and Qwen3 models.

Evidence: RULER and LongBench-Pro benchmarks at context lengths up to 128K tokens.

The Big Picture

Long-context LLM inference is memory-bound. Fitting a 128K-token context in a transformer requires gigabytes of key-value (KV) cache that exceed typical GPU memory. Sparse-attention methods reduce pressure by loading only a selected subset of KV blocks per query. The dominant selection heuristic—loading the top- K blocks by attention mass—has a hidden flaw: it treats unselected blocks as if they carry zero weight, silently biasing the softmax partition function and the attention output.

CompKV revisits this approximation and asks: given that some blocks will not be loaded, what is the best way to account for them? The answer is mean compensation—replacing each unselected block with its mean—combined

with a principled selection criterion derived from the compensation’s residual error.

Why Existing Approaches Fall Short

Eviction methods (H2O, StreamingLLM) permanently discard tokens, making them unrecoverable for later attention layers.

Selection without compensation loads the top- K blocks and zeroes the rest, which underestimates the softmax denominator whenever unselected blocks carry non-trivial mass.

KV quantization reduces per-token size but does not reduce the number of tokens read per attention call.

IO-aware kernels overlap memory transfers with compute but leave the approximation error unaddressed.

None of these methods reason about how unselected context should be summarised to correct the partition function.

Core Mechanism

Mean Compensation. For each unselected block j , CompKV replaces every logit with the block mean $\bar{e}_j$. This is the unique single-number summary that preserves the block’s contribution to the log-partition function to first order—a property that zero-padding and other approximations lack.

Compensation-Aware Priority. After fixing mean compensation, the residual approximation error for block j is bounded by the product of its attention mass m_j (how much it contributes to the output) and its within-block logit standard deviation σ_j (how much individual weights differ from the mean). CompKV selects the blocks with the largest $m_j \cdot \sigma_j$ —those for which mean compensation is most inadequate.

Technical Formulation

Let the KV cache be partitioned into blocks of size b . For block j , denote the attention logits $\mathbf{e}_j \in \mathbb{R}^b$ and the block mean $\bar{e}_j = \frac{1}{b} \sum_i e_{j,i}$.

The log-partition under mean compensation:

$$\log Z_j^{\text{approx}} = \bar{e}_j + \log b$$

which equals the exact value when all within-block logits are equal. The residual error is bounded:

$$\Delta_j \leq m_j \cdot \sigma_j$$

where $m_j = \text{softmax_mass}(\mathbf{e}_j)$ and $\sigma_j = \text{std}(\mathbf{e}_j)$. The CompKV selection priority score is:

$$s_j = m_j \cdot \sigma_j$$

The top- K blocks by s_j are loaded; the rest contribute via their mean.

Learning or Inference Procedure

CompKV is training-free and applies at inference time:

1. A fused GPU kernel computes per-block mass estimates and logit standard deviations using grouped variance projection of the query against pre-computed block mean keys.
2. Blocks are ranked by $s_j = m_j \cdot \sigma_j$; top- K are selected.
3. Selected blocks contribute via exact attention; unselected blocks contribute via their mean, with the partition function adjusted accordingly.

Implementation: BF16 KV cache and FP32 block summaries are kept in pinned CPU memory. A main CUDA stream handles selection-region transfer and exact-attention kernels; an auxiliary stream transfers mean values in parallel, eliminating blocking synchronization.

What the Guarantee Says

Mean compensation guarantees exact log-partition preservation (to first order) when within-block logits are uniform. For non-uniform blocks, the residual error is bounded by $m_j \cdot \sigma_j$ —precisely the quantity used for selection. Under a fixed budget of K blocks, CompKV minimises the total approximation error bound by choosing the blocks for which the bound is largest.

Experimental Findings

- Models: Llama-3.1-8B-Instruct, Qwen3-8B, Qwen3-32B
- Benchmarks: RULER (controlled retrieval and aggregation), LongBench-Pro (realistic long-context tasks), context lengths to 128K tokens
- CompKV outperforms selection-only (top- K by mass), H2O, and StreamingLLM variants consistently across all models and benchmarks
- Gains are largest on aggregation tasks and at tighter budgets (K is small), where the compensation correction matters most

Ablations and Interpretation

Selection criterion: Replacing $m_j \cdot \sigma_j$ with mass-only or variation-only degrades performance, confirming that the joint criterion is necessary.

Compensation type: Mean compensation outperforms zero compensation; the gap grows with block size (larger within-block variation).

IO overhead: The fused kernel architecture hides trans-

fer latency by overlapping streams; throughput is competitive with GPU-only sparse attention at high batch sizes.

Reference

Anonymous et al. **CompKV: Compensation-Aware KV Selection for Long-Context LLM Inference**. arXiv:2609.26300, September 2026.
<https://arxiv.org/abs/2609.26300>

Keywords: diffusion LLMs • KV caching • speculative decoding • inference acceleration

Flash-dLLM

IO-Aware KV Caching and Parallel Decoding for Fast, Memory-Efficient Diffusion LLMs

By Quan Nguyen-Tri, Mukul Ranjan & Zhiqiang Shen (VILA Lab, MBZUAI)

arXiv:2609.26796 • September 2026

An IO-aware fused KV-cache kernel combined with a self-drafting decode strategy eliminates the HBM bandwidth bottleneck that has crippled diffusion LLM inference, delivering $5.1\times$ and $11.0\times$ speedups over the strongest prior method on GSM8K and HumanEval without any model modification.

AT A GLANCE

Problem: Diffusion LLMs require full KV-cache reads at every denoising step; GPU HBM I/O is the dominant bottleneck, not compute.

Why it matters: dLLMs promise parallel token generation, but the IO bottleneck has prevented them from reaching autoregressive LLM speeds.

Method: Fused IO-aware CUDA kernel reduces redundant HBM reads; the dLLM itself serves as drafter and verifier, eliminating auxiliary models.

Result: $5.1\times$ speedup on GSM8K and $11.0\times$ on HumanEval vs. Elastic-Cache; improved memory efficiency at equal quality.

Evidence: Mathematical reasoning and code generation benchmarks on A10G, A100, and L4 GPUs.

The Big Picture

Diffusion language models (dLLMs) generate text by iteratively denoising masked tokens rather than decoding one token at a time. Their non-autoregressive structure should, in principle, allow far more parallelism than autoregressive models. In practice, dLLM inference has remained slow because every denoising step requires reading the entire KV cache—a cost proportional to sequence length that must be paid at each of the T iterations.

Flash-dLLM diagnoses this as an I/O bottleneck: on all tested GPUs, the attention computation in a KV-cache-enabled dLLM is *memory-bandwidth-bound*, not compute-bound. Arithmetic intensity (FLOPs per HBM byte) falls far below the GPU’s compute-to-memory ratio, meaning that the GPU arithmetic units are idle most of the time, waiting for data.

Why Existing Approaches Fall Short

Standard autoregressive KV caching exploits left-to-right structure; dLLMs access the full bidirectional context at every step, so per-step IO cost is proportional to sequence length regardless of generation position.

Elastic-Cache and token eviction reduce cache size but preserve the naive kernel structure: the remaining tokens still require an unfused read-attend-write pipeline that is IO-inefficient.

Auxiliary-model draft-and-verify (as used in speculative decoding for autoregressive models) adds a second model to load and run, increasing memory and latency per step.

Flash Attention for autoregressive models was designed for compute-bound settings; its tiling strategy does not generalise to the full-sequence, bidirectional reads required by dLLMs.

Core Mechanism

IO-Aware Fused Kernel. Flash-dLLM fuses the KV-cache read, attention computation, and output write into a single kernel. Intermediate values are kept in on-chip SRAM, eliminating round-trips to HBM for the intermediate attention maps. The tiling strategy is tuned for the bidirectional, full-sequence access pattern of dLLMs rather than the causal sliding-window pattern of autoregressive models.

Self-Drafting Decode Strategy. Early denoising steps operate at high noise levels and produce coarse but fast predictions. Flash-dLLM uses these as a draft: the KV cache computed in early steps is reused by later steps, and positions where the early draft matches the final refined output avoid recomputation. The dLLM itself is both drafter and verifier—no auxiliary model is loaded.

Technical Formulation

Let $\mathbf{x}^{(t)}$ be the partially denoised sequence at step t , f_θ the dLLM, L the sequence length, d the head dimension, H the number of heads, l the number of layers.

Naive KV-cache IO cost per step:

$$\text{IO}(t) = 2 \cdot L \cdot d \cdot H \cdot l \cdot \text{sizeof}(\text{fp16})$$

Fused-kernel IO cost (tile size B_{SRAM}):

$$\text{IO}_{\text{fused}}(t) = \frac{L \cdot d \cdot H \cdot l}{B_{\text{SRAM}}} \cdot \text{sizeof}(\text{fp16})$$

The draft-and-verify step amortises IO over T_{draft} steps: positions where the draft $\hat{\mathbf{x}}^{(0)}$ at step t matches the verified output at step $t-1$ skip re-attention, contributing a further multiplicative reduction in total IO.

Learning or Inference Procedure

Flash-dLLM requires no training and no model modification:

1. Compile the IO-aware fused CUDA kernel for the target GPU architecture.
2. Run the first T_{draft} denoising steps with the fused kernel; record draft token predictions from early steps.
3. Verify drafts in the remaining $T - T_{\text{draft}}$ steps; reuse KV cache for positions where draft is accepted.
4. Return the fully denoised sequence.

The kernel and draft parameters are tuned once per GPU model; no per-task adaptation is needed.

What the Guarantee Says

The fused kernel is numerically exact: it computes the same attention values as the unfused baseline up to floating-point rounding. Speedup is achieved purely by reducing memory traffic, not by approximation. The self-drafting strategy introduces an optimism bias (assuming later steps will agree with earlier ones), but the verification step guarantees that accepted tokens are correct.

Experimental Findings

- Versus Elastic-Cache (strongest prior baseline): **5.1× speedup** on GSM8K (mathematical reasoning); **11.0× speedup** on HumanEval (code generation)
- Memory efficiency: lower peak HBM usage at equal or better quality
- Code benchmark shows larger gains because longer sequences and sparser denoising trajectories amplify IO savings
- Inference speed approaches autoregressive LLMs with Flash Attention at the same model size

Ablations and Interpretation

Fused kernel alone accounts for 3–5× of the speedup; the self-drafting strategy contributes 1.5–2×.

Draft length T_{draft} : Shorter drafts reduce reuse and speedup; longer drafts risk more corrections but still improve throughput because IO savings outweigh correction costs.

IO profile: On all tested GPUs, attention accounts for over 70% of total inference time with KV caching, confirming the IO-bottleneck hypothesis.

Arithmetic intensity: Before the fused kernel, dLLM attention falls below 10% of the GPU roofline; after, it approaches 60–80%, indicating near-full utilisation of HBM bandwidth.

Reference

Quan Nguyen-Tri, Mukul Ranjan, Zhiqiang Shen. **Flash-dLLM: IO-Aware KV Caching and Parallel Decoding for Fast, Memory-Efficient Diffusion LLMs.** arXiv:2609.26796, September 2026.

<https://arxiv.org/abs/2609.26796>

Keywords: recursive self-improvement • AI research agents
• bi-level optimization • autonomous ML

Recursive Self-Improvement of AI Research Agents

An Outer Agent That Rewrites Its Own Code Beats Human-Engineered Baselines Across Four Held-Out Benchmarks in Eight Days

By Dhruv Srikanth, Bingchen Zhao, Dixing Xu, Yuxiang Wu & Zhengyao Jiang (Weco AI)

arXiv:2609.26457 • September 2026

AIDE², a bi-level system that uses an outer AI agent to autonomously rewrite the scaffolding code of an inner AI research agent, discovered seven successive improvements over an unattended 8-day run—including a 16× prompt compression and a halving of reward hacking—and matched or exceeded a human-engineered production agent on all four out-of-distribution held-out benchmarks.

AT A GLANCE

Problem: AI research agent scaffolding is fixed by human engineers and cannot benefit from the same automated improvement loop that the agent applies to its tasks.

Why it matters: Human R&D yields diminishing returns; a system that autonomously improves its own search policy and memory compresses months of engineering into days.

Method: Outer LLM agent proposes code edits to the inner agent’s scaffolding; edits are accepted only if they improve aggregate performance on a diverse training battery.

Result: 7 successive improvements in 8 days; 16× prompt compression; reward hacking drops from 63% to 34%.

Evidence: Four held-out benchmarks spanning ML engineering, algorithm design, and weather forecasting.

The Big Picture

Autonomous AI research agents such as AIDE (AI Development and Execution) can solve machine learning engineering tasks without human intervention: given a dataset and an objective, the agent proposes, executes, and refines code solutions in an automated loop. These agents have matched human competitors on benchmark challenges. But their *scaffolding*—the code that governs how they search, what they remember, and how they prompt the underlying language model—has always been

designed by human engineers.

This paper asks whether the self-improvement loop can be turned inward. The answer is AIDE²: when the agent’s own scaffolding code is the object of optimization, each accepted rewrite becomes the agent that the next round edits. In 8 unattended days, this loop discovered improvements that human engineers had not found in two years.

Why Existing Approaches Fall Short

Static scaffolding: Existing AI research agents (AIDE, AutoML-GPT, OpenDevin-Science) optimise task solutions but treat their own search policy and prompts as fixed hyperparameters chosen by human experts.

Meta-learning adapts model weights over episodes but requires differentiable objectives and cannot modify discrete code logic.

Neural architecture search and **program synthesis** optimise specific modules in narrow settings; neither is designed to improve a general-purpose agent’s control code.

Human iteration is slow, expensive, and bounded by engineering bandwidth. Moreover, the most obvious improvements have already been incorporated; finding subtler gains requires exhaustive experimentation.

Core Mechanism

Bi-Level Optimization. AIDE² wraps the standard AIDE agent in an outer optimization loop:

- *Inner loop (AIDE):* Given a task and budget, the inner agent proposes, executes, evaluates, and refines code solutions via tree search.
- *Outer loop (AIDE²):* An outer LLM agent inspects the inner agent’s scaffolding code θ , its performance traces, and recent failure modes. It proposes a code edit δ (a patch), spins up a fresh inner agent with $\theta' = \theta + \delta$, evaluates it on approximately 50 diverse tasks, and accepts δ only if performance improves.

Progressive Code Repository. Accepted edits accumulate in a version-controlled repository; each generation’s code is the starting point for the next round. Only about 1 in 10 proposed modifications is accepted.

Technical Formulation

Let π_θ denote the inner agent parameterised by scaffolding code θ , $\mathcal{T}_{\text{train}}$ the training task battery, $\mathcal{T}_{\text{held}}$ the held-out evaluation set.

The outer loop solves:

$$\theta^* = \arg \max_{\theta' \in \mathcal{N}(\theta)} \frac{1}{|\mathcal{T}_{\text{train}}|} \sum_{t \in \mathcal{T}_{\text{train}}} \text{score}(\pi_{\theta'}, t)$$

where $\mathcal{N}(\theta)$ is the set of code edits proposed by the outer LLM. The update rule:

$$\theta \leftarrow \theta' \quad \text{iff} \quad \text{Eval}(\pi_{\theta'}, \mathcal{T}_{\text{train}}) > \text{Eval}(\pi_{\theta}, \mathcal{T}_{\text{train}})$$

The outer agent is itself an LLM that reads θ , performance traces, and recent deltas before producing the patch δ in natural language followed by the code diff.

Learning or Inference Procedure

Outer loop (8 days, fully unattended):

1. Initialise θ_0 from the production AIDE codebase.
2. Outer LLM proposes a code edit δ to θ .
3. Sandboxed inner agent $\pi_{\theta+\delta}$ runs on $\mathcal{T}_{\text{train}}$.
4. If aggregate score improves: commit $\theta \leftarrow \theta + \delta$.
5. Repeat; $\approx 90\%$ of proposals are rejected.

Inner loop (per task):

1. Read task specification and compute budget.
2. Tree search: draft $\rightarrow$ execute $\rightarrow$ score $\rightarrow$ improve.
3. Return best solution within budget.

What the Guarantee Says

No formal convergence bound is given; the search space is open-ended natural-language code. The empirical claim is that a sufficiently diverse training battery (50 tasks, multiple domains) prevents overfitting: improvements that score higher on the training battery also generalise to the four held-out benchmarks—a necessary condition for recursive self-improvement to be meaningful rather than evaluation hacking.

Experimental Findings

- **7 successive improvements** accepted over 8 days from approximately 70 proposed modifications.
- **New search algorithm:** replaced default tree policy with a learned priority function exploiting evaluation feedback.
- **16 $\times$ prompt compression:** a compressed prompting scheme reduces per-step cost and enables longer search horizons.
- **Reward hacking:** self-policing emerged naturally; hacking rate dropped from 63% to 34% on a held-out benchmark.
- **4 held-out benchmarks** (ML engineering, algorithm engineering, physics forecasting, and one additional ML benchmark): the evolved agent matches

or exceeds the human-engineered production AIDE on all four.

Ablations and Interpretation

Task diversity: Narrowing the training battery to a single domain causes held-out generalisation to degrade significantly; diversity is essential.

Outer model size: Larger outer LLMs produce higher-quality proposals and higher acceptance rates, but even smaller models drive improvement given sufficient budget.

Reward hacking: The self-policing behavior was not explicitly incentivised; it emerged because reward hacking lowered aggregate evaluation scores and the outer loop penalised it implicitly—an encouraging sign that the optimisation pressure is well-aligned.

Gain decomposition: Prompt compression contributes $\approx 40\%$ of the total held-out improvement; the new search policy $\approx 35\%$; memory mechanisms $\approx 25\%$.

Reference

Dhruv Srikanth, Bingchen Zhao, Dixing Xu, Yuxiang Wu, Zhengyao Jiang. **Recursive self-improvement of AI research agents**. arXiv:2609.26457, September 2026.
<https://arxiv.org/abs/2609.26457>

Keywords: visual token pruning • multimodal LLMs • conditional removability • inference efficiency

CoRePrune

From Token Importance to Conditional Removability: Rethinking Visual Token Pruning in Multimodal Large Language Models

By Shengli He et al.

arXiv:2609.26484 • September 2026

Token importance and redundancy are poor proxies for removability because they ignore representation depth and deletion-set context; CoRePrune’s progressive depth-aware perturbation tracking and set-conditioned refinement retain 90.3% of dense-model performance with 128 visual tokens while cutting prefill time by 51% across five multimodal LLM backbones.

AT A GLANCE

Problem: Visual token pruning methods score tokens by importance or redundancy but miss that removability is conditioned on transformer depth and the surrounding deletion set.

Why it matters: Incorrect removability estimates cause unexpected quality drops at aggressive budgets; fixing the estimate recovers performance.

Method: Two-stage CoRePrune: progressive perturbation-aware visual pruning across depth, followed by set-conditioned refinement after visual-text interaction.

Result: 90.3% dense-model performance with 128 tokens; 51.0% prefill speedup on five MLLM backbones.

Evidence: Standard image, high-resolution, and video benchmarks across LLaVA-1.5, InternVL2, LLaVA-HR, and VideoLLM.

The Big Picture

Multimodal large language models (MLLMs) convert images into sequences of visual tokens—typically hundreds to thousands—that dominate the cost of the prefill phase. Training-free token pruning addresses this by scoring each visual token and discarding the lowest-scoring ones before or during the forward pass. The appeal is obvious: no fine-tuning, no architecture change, just a cheaper forward pass.

The trouble is that existing scoring functions—attention weight, activation norm, pairwise similarity—treat removability as a static per-token property. CoRePrune shows that it is not. Removing the same token at layer 5 versus layer 20 causes substantially different output perturba-

tions because the representations have been increasingly fused with context. Removing a token when 100 others are already gone has a different effect than removing it from the full set because the remaining tokens have compensated for prior deletions. These two dependencies—*depth* and *deletion set*—are precisely what existing methods ignore.

Why Existing Approaches Fall Short

Importance-based methods (attention score, gradient norm) score tokens in isolation and at a single layer; they cannot predict how importance changes as representations propagate through depth.

Redundancy-based methods (cosine similarity) identify tokens that are similar to their neighbours, but spatial similarity at one layer does not imply similarity after additional self-attention mixing.

Static depth assignment: Most methods apply pruning at a fixed layer (early or late), ignoring that the same token may be safe to remove at depth 20 but harmful to remove at depth 5.

Set-agnostic scoring: All existing methods score each candidate independently, ignoring that removing token v_i with v_j already gone may have a very different effect than removing v_i from the full set.

Core Mechanism

Conditional Removability. CoRePrune operationalises removability as the downstream perturbation caused by removing a token or set of tokens, measured at the current depth under the current deletion set. This quantity is tracked dynamically rather than estimated once at a fixed layer.

Stage 1: Progressive Perturbation-Aware Visual Pruning (PPVP). As the forward pass proceeds through successive transformer layer blocks, PPVP re-estimates the perturbation each candidate removal would cause. A token whose removal perturbation *shrinks* as depth increases has been compensated for by the model’s own representation mixing—it is safe to remove. Tokens whose perturbation grows are promoted in priority.

Stage 2: Set-Conditioned Refinement (SCR). After the first visual-text interaction layer (where visual and text tokens first attend to each other), SCR re-evaluates each remaining candidate under the *current* deletion set. Tokens that now provide net benefit given what has already been removed are recovered; tokens whose marginal contribution has collapsed are added to the deletion set.

Technical Formulation

Let $V = \{v_1, \dots, v_N\}$ be the visual token sequence and $f_l(S)$ the representations at layer l for token set S .

Perturbation at depth l for deletion set D :

$$P_l(D) = \|f_l(V \setminus D) - f_l(V)\|_F$$

PPVP priority update for candidate v_i :

$$\text{safe at } l_2 \iff P_{l_2}(\{v_i\}) < P_{l_1}(\{v_i\}), \quad l_2 > l_1$$

SCR marginal contribution under current deletion set D at layer l^* :

$$r(v_j; D) = P_{l^*}(D) - P_{l^*}(D \cup \{v_j\})$$

Recover v_j if $r(v_j; D) > 0$; add v_j to D if $r(v_j; D) \ll 0$.

Learning or Inference Procedure

CoRePrune is **training-free** and runs during the prefill phase:

1. Forward pass to layer l_1 : compute initial perturbation estimates.
2. Continue to layer $l_2 > l_1$: update estimates via PPVP; rank candidates.
3. Tentatively remove lowest-priority tokens to meet the token budget.
4. Continue to visual-text interaction layer l^* .
5. Apply SCR: recover tokens with $r(v_j; D) > 0$; finalise deletion set.
6. Continue forward pass with the pruned set; generate output.

Overhead: two additional perturbation probes during prefill, negligible relative to the attention savings from the reduced token count.

What the Guarantee Says

PPVP provides a constructive argument: if $P_{l_2}(\{v_i\}) < P_{l_1}(\{v_i\})$, the model has already compensated for v_i 's absence by depth l_2 , making removal safe. SCR provides a greedy local-optimum guarantee: after SCR, no single token swap (recovery or addition) would reduce the deletion-set perturbation.

Experimental Findings

- **5 MLLM backbones:** LLaVA-1.5, InternVL2, LLaVA-HR, VideoLLM, and one additional model
- At **128 visual tokens:** CoRePrune retains **90.3%** of dense-model performance (vs. 83–86% for prior methods) and reduces aggregate prefill time by **51.0%** (vs. 45–48% prior)

- At 64 tokens: advantage over baselines grows due to stronger deletion-set interaction effects
- Largest gains on global reasoning tasks (VQA, video QA); smaller gains on local patch tasks where interaction effects are weaker

Ablations and Interpretation

PPVP alone removes the depth-static assumption and recovers half the total gain over attention-based baselines.

SCR alone removes the set-agnostic assumption and recovers the other half. The joint gain exceeds the sum, revealing non-linear interaction.

PPVP probe depth: Tracking across ≥ 3 layer blocks is necessary; fewer blocks converges to depth-static behaviour.

SCR timing: Applying SCR before visual-text interaction or at the final layer both degrade performance; the visual-text fusion layer is the optimal intervention point.

Reference

Shengli He et al. **From Token Importance to Conditional Removability: Rethinking Visual Token Pruning in Multimodal Large Language Models.** arXiv:2609.26484, September 2026.
<https://arxiv.org/abs/2609.26484>

Keywords: floating-point precision • greedy decoding • LLM reproducibility • numerical divergence

Greedy Decoding Is Not Precision-Invariant

Cross-Precision Output Divergence in Large Language Model Inference

By Gaoyuan Du, Anam Nawaz Khan, Rex Zhou, Xiaoyang Liu, Deepayan Chakrabarti, Fnu Suya & Xueping Li
(UTK • University of Chicago • Amazon • UT Austin)

arXiv:2609.26621 • September 2026

BF16 and FP16 greedy decoding from the same model on the same hardware diverges for 49–100% of prompts across six models and three benchmarks; the divergence is predicted not by deep-layer error accumulation but by the top-two logit margin at the language model head, and selective FP32 recomputation of that head at tight-margin steps recovers +22–36 pp exact agreement at only $\pm 4\%$ latency overhead.

AT A GLANCE

Problem: Greedy decoding is assumed to be deterministic given a fixed model and prompt, but BF16 and FP16 produce different outputs on identical hardware at a surprisingly high rate.

Why it matters: Cross-precision divergence confounds evaluations, complicates deployment, and undermines reproducibility guarantees.

Method: Empirical error-propagation analysis traces divergence to the top-two logit margin at the LM head; selective FP32 recomputation at low-margin steps corrects most divergences.

Result: +22–36 pp exact agreement recovered on A10G at $\pm 4\%$ latency overhead; approach validated across three GPU architectures.

Evidence: Six models (1.1B–7B, four families), three benchmarks (MMLU, TriviaQA, GSM8K).

The Big Picture

Large language model evaluations routinely compare numbers across frameworks, hardware generations, and model serving stacks. An implicit assumption underlies all such comparisons: greedy decoding is *deterministic* given a fixed model. Feed in the same model weights, the same prompt, and the same hardware, and you get the same output.

This paper breaks that assumption. It shows that switching between BF16 and FP16—two floating-point formats

in equally widespread use—produces different greedy outputs from the same model on the same GPU for 49 to 100% of prompts, depending on the model and benchmark. The divergence is not marginal: a single token flip typically cascades into a fully different response trajectory.

The culprit is not the 22 layers of accumulated numerical error that one might expect. It is a *local* phenomenon: whether the top-two logits at the language model head are separated by more than the cross-precision perturbation at that point in the computation.

Why Existing Approaches Fall Short

Same-precision nondeterminism (parallelism-order variation within BF16 or FP16) is well-studied and mitigated by deterministic CUDA kernels. It does not explain *cross-precision* divergence, which is systematic and reproducible.

Full FP32 inference eliminates the problem but doubles memory usage and costs roughly $1.8\times$ in latency—impractical for production.

Precision-agnostic evaluation (reporting numbers without specifying precision) silently inflates or deflates benchmark scores by comparison-format mismatches.

No targeted, low-cost mitigation existed before this work.

Core Mechanism

Systematic Characterization. The authors evaluate six LLMs (1.1B–7B, four families) with greedy decoding in BF16 and FP16 on MMLU, TriviaQA, and GSM8K on identical A10G hardware. For each prompt, they record whether the full trajectory diverges.

Error-Propagation Analysis. For each diverging prompt, they trace the first-flip location and measure accumulated numerical error at each layer and at the LM head. The finding: 22 layers of body error do not differentiate flipping from non-flipping steps. Divergence is determined at the LM head.

Selective FP32 Head Recomputation. When the top-two logit margin at the LM head falls below a threshold τ , the head matrix multiplication is re-run at FP32 for that decoding step. Steps with large margins are left at native precision.

Technical Formulation

Let $\mathbf{h} \in \mathbb{R}^d$ be the last-layer hidden state and $\mathbf{W} \in \mathbb{R}^{V \times d}$ the LM head weight matrix. Logits under precision $p \in \{\text{bf16}, \text{fp16}\}$:

$$\mathbf{z}^{(p)} = \mathbf{W}^{(p)} \mathbf{h}^{(p)} + \boldsymbol{\epsilon}^{(p)}$$

where $\boldsymbol{\epsilon}^{(p)}$ is accumulated rounding error.

A token flip occurs when:

$$\arg \max \mathbf{z}^{(\text{bf16})} \neq \arg \max \mathbf{z}^{(\text{fp16})}$$

Empirically, the flip probability is well-predicted by:

$$\Pr[\text{flip}] \approx \Pr[\Delta_{\text{top2}} < \|\epsilon^{(\text{bf16})} - \epsilon^{(\text{fp16})}\|_{\text{top2-dir}}]$$

where $\Delta_{\text{top2}} = z_{(1)} - z_{(2)}$ is the top-two margin.

Selective FP32 trigger:

$$\text{recompute at FP32} \iff \Delta_{\text{top2}} < \tau$$

Learning or Inference Procedure

No training is required. The mitigation applies at each decoding step:

1. Run the transformer forward pass in native precision (BF16 or FP16).
2. Compute logits and the top-two margin Δ_{top2} .
3. If $\Delta_{\text{top2}} < \tau$: re-run the LM head matmul in FP32; use FP32 logits for this step.
4. Select the greedy token; proceed.

The threshold τ is calibrated per hardware on a small validation set.

What the Guarantee Says

Selective FP32 head recomputation is an empirical, not worst-case, mitigation. It cannot guarantee identical outputs to any reference because cross-precision error accumulates throughout the forward pass, not only at the head. However, because the head is the single decision point and body error is shown not to be predictive, covering tight-margin head steps recovers most of the divergence. The remaining divergence (about 10–30%) stems from body-precision differences that the head-only fix does not address.

Experimental Findings

- **Divergence rate:** 49–100% of prompts across 6 models and 3 benchmarks; rate varies by model family and task type.
- **Cascade effect:** A single first-token flip cascades into full trajectory divergence in >80% of cases.
- **Error-propagation:** Accumulated body error over 22 layers does not predict flipping; top-two margin at the LM head is the dominant predictor.
- **Selective FP32 on A10G:** +22–36 pp exact-agreement recovery at $\pm 4\%$ latency overhead.
- **L4 and A100:** +12–21 pp recovery (smaller because baseline divergence rates differ by GPU architecture).

Ablations and Interpretation

Threshold sensitivity: Agreement recovery is robust across a $5\times$ range of τ ; the optimal value balances coverage (catching flip-prone steps) against unnecessary FP32 overhead.

Full FP32: Achieves near-zero divergence but at $2\times$ memory and $\sim 1.8\times$ latency—impractical for production serving.

Body recomputation: Re-running the full transformer in FP32 at flip-predicted steps adds disproportionate overhead for marginal additional gain.

Vocabulary size effect: Larger vocabularies have smaller average top-two margins (more competing candidates per step) and higher divergence rates, suggesting the problem worsens as vocabulary size grows.

Task type: Open-ended generation (TriviaQA) shows higher divergence than constrained generation (MMLU multiple-choice), as cascading divergence has more opportunity to propagate in longer unconstrained outputs.

Reference

Gaoyuan Du, Anam Nawaz Khan, Rex Zhou, Xiaoyang Liu, Deepayan Chakrabarti, Fnu Suya, Xueping Li. **Greedy Decoding Is Not Precision-Invariant: Cross-Precision Output Divergence in LLM Inference.** arXiv:2609.26621, September 2026.

<https://arxiv.org/abs/2609.26621>